

L-Diff: Fast and Efficient Pretraining–Finetuning Framework for Differential Transformer-based Language Models

Yujeong Son*, Eunseok Cho[†]

*Dept. of Electrical Engineering, UNIST, Ulsan, Korea

[†]Artificial Intelligence Graduate School, UNIST, Ulsan, Korea

{syj4739, eric754}@unist.ac.kr

Abstract

The scalability of Large Language Models (LLMs) has fueled rapid advances across a wide range of AI applications, as ever-larger models with longer context windows continue to push the frontier of performance. Yet, training these models efficiently and stably under realistic compute constraints remains challenging, especially when attempting to maintain strong contextual reasoning and alignment. Within this landscape, Differential Transformers provide a promising alternative by suppressing attention noise and focusing on informative contextual changes, but their difference-based attention mechanism is not directly compatible with conventional pretraining and fine-tuning pipelines designed for standard softmax attention, often leading to unstable optimization and suboptimal transfer when applied naively. This paper presents L-Diff, a scalable and efficient training framework built on Differential Transformers (DT) for faster and efficient training. L-Diff combines an improved Differential Transformer architecture with FlashAttention-2 kernels, and fused GPU kernels to increase memory efficiency and training throughput for long-context modeling. The training pipeline consists of pretraining method including tensor alignment, followed by a two-stage alignment process using Supervised Fine-Tuning and Direct Preference Optimization. Experiments on a 159M-parameter causal language model show that L-Diff achieves about 30% lower early-stage loss and reduces total training time to roughly 28% of a vanilla Transformer baseline, while maintaining stable gradients and demonstrating the effectiveness of the proposed architectural and methodological optimizations under single-GPU constraints.¹

Keywords— Differential transformer, Pretraining, Fine-Tuning, Supervised Fine-Tuning (SFT), Direct Preference Optimization (DPO)

1 Introduction

The scalability of Large Language Models (LLMs) has driven rapid progress across modern AI systems. However, achieving efficient training and robust context modeling remains a significant challenge, particularly under constrained computational resources. Recent architectural advances such as Mixture-of-Experts (MoE) (Cai et al., 2025) and the Differential Transformer (Ye et al., 2025) have emerged as promising solutions. MoE models activate only a small subset of parameters per token, enabling substantial model scaling without a proportional increase in computation. In parallel, the Differential Transformer improves upon standard Softmax attention by incorporating a differential attention mechanism that suppresses redundant activations and filters out attention noise, thereby enabling more precise context modeling.

While numerous techniques such as transfer learning, parameter-efficient adaptation, and Mixture-of-Experts (MoE) architectures have shown strong performance gains in standard Transformer models, directly applying these methods to Differential Transformers remains non-trivial. Differential attention updates rely on capturing changes between token representations, which introduces sensitivity to distributional shifts during transfer learning. As a result, naively reusing weights pretrained under conventional Softmax attention can lead to unstable optimization or degraded contextual representations.

Similarly, MoE architectures pose integration challenges. Expert routing policies are typically optimized around absolute token embeddings, whereas Differential Transformers operate on difference-based features that exhibit different activation patterns. This mismatch can interfere with expert selection dynamics, potentially causing unbalanced routing or expert underutilization. Moreover, MoE’s already high memory footprint exacerbates the difficulty of maintaining stability within the differential attention mechanism, especially under limited-resource settings.

Other augmentation techniques—such as long-context scaling (Xiong et al., 2023), retrieval-based memory (Kuang et al., 2024), and speculative decoding (Leviathan et al., 2023)—also face compatibility issues. These methods often assume that the model’s underlying attention distribution is smooth and globally normalized, whereas differential attention emphasizes local relational changes. As a consequence, many of these enhancements require substantial architectural adaptation or rethinking of positional dynamics to function reliably within Differential Transformer frameworks.

To overcome these limitations, we propose L-Diff, an efficient training framework that integrates architectural optimization with a structured learning pipeline. L-Diff incorporates a Differential Transformer enhanced with FlashAttention-2 kernels to maximize memory efficiency and GPU throughput. Beyond architectural improvements, we introduce a training strategy that accelerates early convergence and improves alignment: a robust pretraining phase followed by a two-stage instruction-tuning process consisting of Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO).

¹This work is results for the final project of UNIST AI502 Principles of Deep Learning course in 2025 Fall semester.

This pipeline enables the model to quickly reduce training loss—achieving 30% lower loss in early steps—and significantly shortens the overall training time to 28% compared to baseline approaches.

In this paper, we make the following contributions:

- **Fast & Efficient Differential Transformer(DT) Implementation:** We implement a high-performance DT integrated with custom FlashAttention kernel. This design significantly optimizes memory usage and throughput for long-context training.
- **Optimized Pretraining Method:** We integrated a GPT-4-based tokenizer with a vocabulary padded to the nearest multiple of 64 for Tensor-Core-efficient training, and designing the learning rate scheduler used in the pretraining pipeline.
- **Two-Stage Fine-tuning Pipeline (SFT & DPO):** We propose a comprehensive finetuning phase that sequentially applies Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO). This pipeline facilitates deeper learning, allowing the model to effectively refine its policy and adhere to human preferences.

2 Backgrounds

2.1 Standard and Differential Attention

The standard Transformer architecture relies on the Scaled Dot-Product Attention mechanism, which computes the compatibility function between queries and keys to assign weights to values. Formally, given query Q , key K , and value V , the attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

While effective, this mechanism often attends to irrelevant context, introducing noise into the context aggregation process.

To address this, the **Differential Transformer**(Ye et al., 2025) introduces a differential attention mechanism. It splits the query and key vectors into two groups and computes two separate attention maps. The final output is derived from the difference between these two maps:

$$\text{DiffAttn} = (\text{Attn}_1 - \lambda \odot \text{Attn}_2)V \quad (2)$$

Here, λ is a learnable scalar that controls the magnitude of the subtracted attention map. This formulation allows the model to cancel out common-mode noise and focus on more distinctive features within the context, effectively enhancing the signal-to-noise ratio of the attention mechanism. Compared to standard Transformers, Differential Transformers offer several advantages. First, they improve optimization stability by reducing activation magnitude and mitigating gradient explosion in early training. Second, they enhance sample efficiency, as difference-based updates allow the model to focus on meaningful contextual shifts rather than re-encoding static information. Finally, the structure is computationally lightweight and compatible with modern attention kernels such as FlashAttention, enabling faster training without additional architectural overhead. These properties make Differential Transformers an appealing alternative to conventional Transformer designs, especially in resource-constrained or efficiency-critical settings.

2.2 Efficient Training Optimization

Scaling Large Language Models requires handling massive computational loads. To mitigate this, we incorporate several hardware-aware optimization techniques.

Flash Attention 2 and Variable Length Processing.

We employed FlashAttention-2 (Dao, 2023) to resolve the quadratic memory bottleneck of standard attention. Instead of materializing the full attention matrix, Flash Attention computes the output by decomposing the query, key, and value matrices into blocks that fit within the GPU’s fast SRAM. This tiling mechanism can be formalized as:

$$O_i = \sum_j \text{softmax} \left(\frac{Q_i K_j^T}{\sqrt{d}} \right) V_j \quad (3)$$

where Q_i, K_j, V_j denote the partitioned blocks. This approach reduces memory I/O complexity to linear, significantly accelerating training. Furthermore, we integrated a custom unpadding strategy to handle variable-length sequences, eliminating redundant computations on padding tokens.

Hardware-Efficient Kernel Fusion via Liger Kernel.

To maximize training throughput and overcome the memory bandwidth limitations inherent in standard PyTorch implementations, we incorporated Liger Kernel(Hsu et al., 2025). This library leverages Triton-based kernel fusion to consolidate sequences of element-wise operations into monolithic CUDA kernels. We specifically replaced the standard implementations of RMSNorm, SwiGLU, and Cross Entropy Loss with their fused counterparts. By keeping intermediate activations within the GPU’s fast on-chip registers (SRAM) rather than repeatedly accessing High Bandwidth Memory (HBM), this approach significantly minimizes memory read/write overhead and reduces kernel launch latency, resulting in higher GPU utilization.

2.3 Fine-tuning Methods

Supervised Fine Tuning (SFT). Supervised fine-tuning is commonly used as the initial stage of instruction tuning for large language models(Pareja et al., 2024). In SFT, models are trained on instruction–response pairs to learn task-specific behaviors and to align their outputs with high-quality demonstrations. This process enables models to establish a foundational capability for following instructions, reasoning over structured tasks, and producing coherent responses. However, SFT only teaches models to imitate reference outputs and does not incorporate human preference signals, which often leads to overly deterministic or suboptimal behaviors when exposed to ambiguous or open-ended tasks.

Parameter-Efficient Fine-Tuning (PEFT). Fine-tuning full model parameters is often prohibitively expensive(Xu et al., 2023). We adopt **Low-Rank Adaptation (LoRA)** for the supervised fine-tuning (SFT) stage. LoRA freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture. For a weight matrix $W_0 \in \mathbb{R}^{d \times k}$, the update is represented as $W_0 + \Delta W = W_0 + BA$, where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ are low-rank matrices ($r \ll \min(d, k)$). This approach allows us to adapt the Differential Transformer to instruction-following tasks with negligible computational overhead.

Proximal Policy Optimization (PPO). Reinforcement Learning from Human Feedback (RLHF) traditionally relies on Proximal Policy Optimization (PPO)(Schulman et al., 2017) to optimize model outputs according to a reward model learned from human preferences. PPO constrains policy updates through a clipping objective, preventing drastic deviations from the reference model and stabilizing training. While effective, PPO introduces significant complexity: it requires (1) a separately trained reward model, (2) careful tuning of KL-penalty coefficients to avoid mode collapse, and (3) high computational overhead due to its reinforcement learning formulation. These factors make PPO difficult to scale efficiently, especially in resource-limited environments.

Direct Preference optimization (DPO). Direct Preference Optimization (Rafailov et al., 2023) replaces the RLHF pipeline with a simplified, end-to-end objective that directly optimizes the probability ratio between preferred and dispreferred responses. DPO removes the need for a reward model and avoids policy-gradient instability by framing preference alignment as a supervised objective rather than an RL problem. As a result, DPO exhibits better training stability, lower computational cost, and more predictable convergence behavior. Empirically, DPO often produces more aligned and natural responses compared to PPO while requiring significantly fewer engineering components.

3 Proposed Methods

3.1 Architecture

To maximize computational throughput and memory efficiency, we integrated FlashAttention kernels into DT architecture. While the Differential Attention mechanism effectively eliminates noise by subtracting two attention maps, it typically incurs higher memory overhead due to dual-head computations. By leveraging FlashAttention’s IO-aware algorithm, we minimized the frequent read/write operations between GPU HBM and SRAM. This integration allows the model to compute the differential subtraction logic directly within the fused kernel, significantly reducing the materialization of large $N \times N$ attention matrices. Consequently, our implementation achieves a substantial increase in training speed and handles longer sequences more effectively than standard DT implementations, shown in section 4.

3.2 Pretrain Method

We conducted pre-training on the Wikitext-2 dataset using a Causal Language Modeling (CLM) objective. In this framework, the model learns to predict the subsequent token based on the preceding context sequence. To ensure robust convergence and computational efficiency within a single-GPU environment, we established the following training strategies:

3.2.1 Method 1: Tensor Alignment

We employed the GPT-4 tokenizer to leverage its high compression ratio for efficient subword processing. To further optimize computational throughput on GPU hardware, we combined tokenizer–vocabulary optimization and a tailored learning rate schedule. The model employs a GPT-4–based tokenizer whose vocabulary is padded to the nearest multiple of 64, enabling tensor-core-friendly matrix dimensions and reducing overhead in embedding and output projection layers.

3.2.2 Method 2: Optimization and Scheduling

The training process utilized the AdamW optimizer with a weight decay of 0.1 to mitigate overfitting, which is a critical concern given the limited size of the Wikitext-2 corpus. We adopted a Cosine Annealing learning rate scheduler accompanied by a 200-step linear warmup phase. To address the data scarcity of the target dataset, we employed an aggressive learning rate strategy with a high peak value (6×10^{-4}). This approach allows the model to rapidly capture dominant linguistic patterns and escape local minima during the early stages of training before the learning rate decays. Fig 1 shows the change of learning rate across the steps.

3.3 Fine-tuning

Based on our model following with the pretraining phase, a two-stage fine-tuning strategy is employed. The first stage consists of Supervised Fine-Tuning (SFT), utilizing instruction–output pairs to establish the model’s fundamental response capabilities. In this phase, the model learns to produce ground-truth or exemplary responses, thereby enhancing task-specific

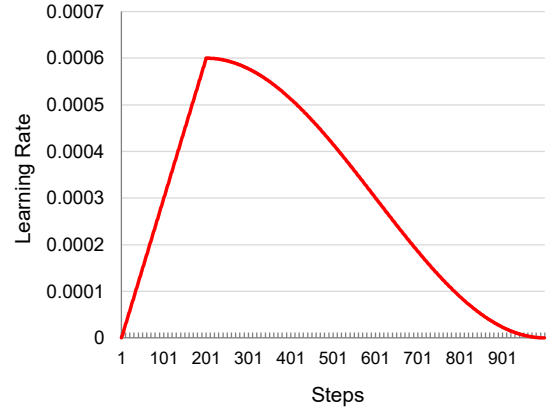


Figure 1: Change of the learning rate by the scheduler.

performance. By performing a full-parameter update, the model is adapted to new data distributions, enabling it to handle domain-specific queries and follow user instructions effectively. Following SFT, we applied Direct Preference Optimization (DPO). DPO utilizes a dataset comprising prompts, multiple response candidates, and their corresponding preference labels to align the model with user-preferred outcomes. Unlike SFT, which focuses on identifying the correct answer, DPO optimizes the model through relative comparisons between responses. This feedback-driven process ensures that the model is fine-tuned toward human preferences, resulting in the generation of more natural and contextually superior responses.

3.3.1 Phase 1: Supervised Fine Tuning

In the SFT phase, the SlimOrca dataset (Lian et al., 2023) was selected to maximize the model’s logical reasoning and instruction-following performance. As a high-quality dataset filtered from GPT-4 generated responses, SlimOrca provides a dense collection of “Chain-of-Thought” examples, which is crucial for helping lightweight models acquire complex problem-solving skills. To ensure efficient fine-tuning, we employed LoRA (Hu et al., 2021) with the rank (r) set to 64 and alpha (α) to 128. These values were chosen to provide a higher capacity for the model to capture and represent intricate task-specific patterns that standard lower ranks might miss. Furthermore, a learning rate of $2e-4$ was applied to balance stability and convergence speed, allowing the model to effectively transfer its pretrained knowledge to the target domain while maintaining the integrity of the original weights.

3.3.2 Phase 2: Direct Preference Optimization

Following the SFT phase, we employed the intel’s orca dpo pairs dataset to align the model’s outputs with human preferences through DPO. This dataset, curated by Intel from the Orca corpus (Mukherjee et al., 2023), consists of high-quality preference pairs that effectively guide the model in distinguishing and selecting more helpful and natural responses. To ensure consistency across the fine-tuning stages and provide sufficient representational capacity, the LoRA hyperparameters were maintained at $r = 64$ and $\alpha = 128$. This allows the model to capture the subtle nuances of preference-based weight adjustments without being bottlenecked by low-rank constraints. Furthermore, a refined learning rate of $5e-6$ was applied. This lower rate was specifically chosen to prevent catastrophic forgetting of the instruction-following capabilities established during SFT, ensuring a stable and precise optimization process that strictly adheres to the preferred response distributions.

Category	Specification
GPU	NVIDIA RTX A6000
Parameter Size	159M
Pretraining Dataset	WikiText-2
Training Precision	BFloat16
Max Sequence Length	2048

Table 1: Experimental Environments

4 Experiments

This section presents a comprehensive evaluation of our optimized model and the proposed training methodologies. We analyze the performance across three key dimensions: the rate of loss convergence, overall training efficiency (time), and model accuracy across various standard benchmarks. All experiments were conducted on an NVIDIA RTX A6000 GPU. We utilized a model architecture with 159 million parameters. The training was performed using the WikiText-2 dataset, and all models were trained using the BFloat16 (BF16) precision format. Experimental environments are also shown in Table 1.

4.1 Ablation Study

This section presents the results of a comprehensive ablation study designed to validate the individual and combined efficacy of our implemented Differential Transformer (DT) architecture and the novel pretraining methodology. Our primary analysis focuses on the convergence behavior during the initial training epochs, which serves as a critical indicator of optimization efficiency and stability.

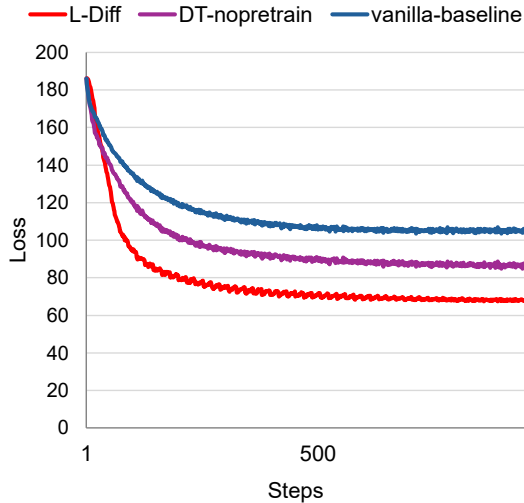


Figure 2: Loss of the first 1000 steps during the pretraining.

As illustrated in Figure 2, we compared the loss reduction rate across three distinct configurations during the first 1,000 pretraining steps. Specifically, the analysis begins by examining the difference between the Vanilla Transformer baseline (represented by the blue curve) and our core implementation, the DT integrated with the FlashAttention kernel (represented by the purple curve). A direct comparison reveals that our implemented model achieves a demonstrably faster rate of loss reduction. In detail, L-Diff achieved a 30% lower loss at steps 100 and 200, which can be considered the early stage of training. This outcome confirms that the combined architectural optimization—the Differential Attention structure leveraged

Configuration	Training time (m)
Vanilla Transformer	~ 180
DT+nopretrain	~ 60
L-Diff	~ 50

Table 2: Training Time Comparison

by the highly efficient FlashAttention kernel—significantly improves optimization efficiency and accelerates the training process beyond the standard baseline.

Furthermore, we assess the contribution of the proposed pretraining technique (detailed in Section 3) by comparing the purple curve (DT + FlashAttention) with the red curve (DT + FlashAttention + Proposed Pretraining). The results clearly indicate that the application of our methodology leads to a further, substantial acceleration in the speed of loss convergence. This finding validates the effectiveness of our proposed pretraining technique, demonstrating its synergistic role in enhancing the DT model’s ability to assimilate knowledge rapidly and efficiently during the early stages of training.

Table 2 shows the approximate time required to complete the training phase for the same configurations analyzed in the loss convergence study. As evident from table 2, these aforementioned optimization techniques have a clear positive impact on the overall training time, confirming their effectiveness in enabling significantly faster model training.

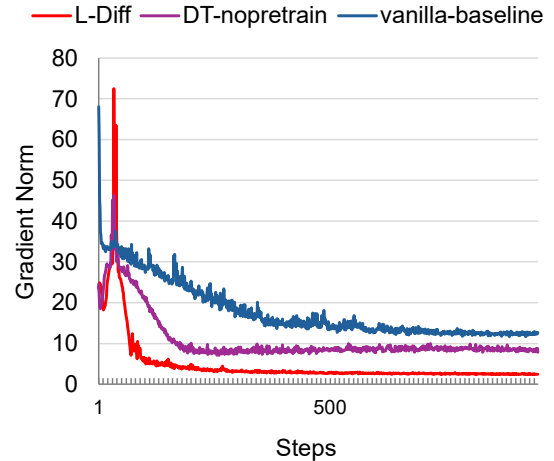


Figure 3: Gradient norm of the first 1000 steps during the pretraining.

Fig 3 illustrates the gradient norm dynamics for each model configuration under the optimization and scheduling strategies specified in Section 3.2. Alongside the Loss Curve, the stability of the gradient norm is a crucial indicator of training reliability. We observe that the gradient norm remains stable across all settings, primarily due to the application of the AdamW Optimizer with a weight decay of 0.1 and the 200-step linear warmup phase. This stability effectively prevents gradient explosion during the critical initial stages. Crucially, this robustness is maintained despite the aggressive high peak learning rate $6e-4$ utilized to address data scarcity, suggesting that the computational efficiency derived from the Tensor Alignment strategy and the FlashAttention kernel significantly enhances the robustness of the optimization process. Furthermore, the DT+FlashAttention model consistently maintains a lower and more uniform gradient norm compared to the Vanilla Transformer, demonstrating that our architectural improvements fundamentally contribute to enhanced training

stability beyond mere speed acceleration.

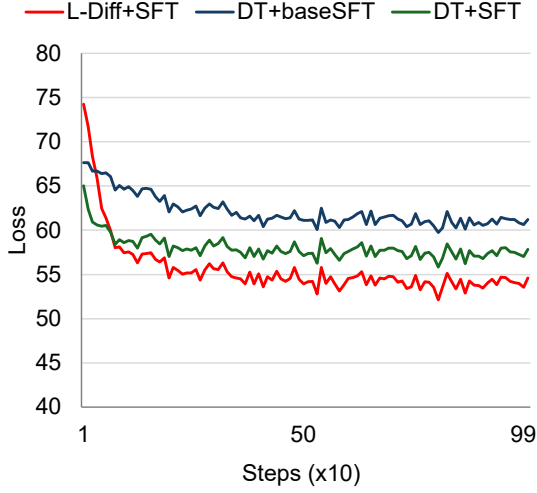


Figure 4: Loss of the first 1000 steps during the supervised fine tuning.

Fig 4 illustrates the results following the SFT phase, confirming the effectiveness of our optimized methodologies across three distinct configurations. The blue curve represents the baseline SFT strategy applied to the DT + FlashAttention model without incorporating the proposed pretraining technique. In contrast, the green curve shows the results when the same architectural base is subjected to the optimized SFT method detailed in Section 4.3.1. A direct comparison between the green and blue curves reveals a demonstrably lower loss for the green configuration, confirming the significant positive impact of our SFT optimization technique alone. Ultimately, the red curve, representing the full L-Diff pipeline before the fine tuning—which integrates the pretraining technique with optimized SFT—yields the best performance, achieving the lowest loss overall. This result conclusively underscores the synergistic effect of combining both architectural optimizations and methodological improvements throughout the entire training lifecycle.

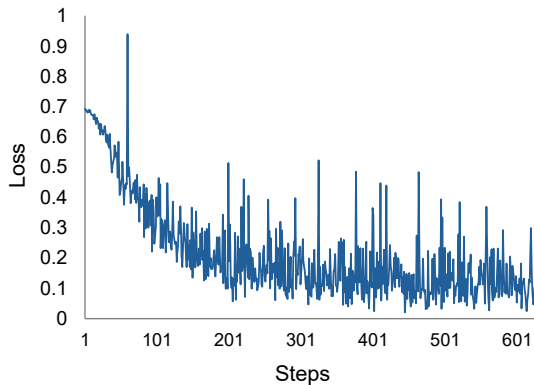


Figure 5: Loss of the first 625 steps during the direct preference optimization.

Following the SFT phase, the L-Diff pipeline incorporates DPO to enable the model to generate more natural and highly preferred responses. Fig 5 illustrates the DPO loss curve when applied to the SFT-trained model (which corresponds to the best-performing configuration, the red curve in Figure

4). This curve demonstrates a distinct further reduction in loss, confirming that the DPO phase successfully enhances performance and aligns the model more closely with desired human preferences.

4.2 Benchmark Results

Benchmarks	Score
GSM8K	0
MMLU	0.24
ARC Easy	0.26
HellaSwag	0.24

Table 3: Benchmark scores for GSM8K, MMLU, ARC Easy, and HellaSwag

In this section, we present the benchmark results obtained from evaluating the model on multiple tasks, including GSM8K, MMLU, ARC Easy, and HellaSwag. The corresponding scores are shown in Table 3. Unfortunately, the benchmark performance was not satisfactory. Due to limited computational resources and the resulting excessively long training time, we were unable to train a model with a sufficiently low perplexity, which consequently led to the sub-optimal benchmark results. As future work, we plan to scale up using larger datasets and models with more parameters to achieve stronger performance on real benchmark tasks.

5 Conclusion

In this paper, we proposed L-Diff, a lightweight, efficient training framework that integrates an enhanced Differential Transformer architecture with FlashAttention kernels, pre-training pipeline, and a two-stage fine-tuning strategy. For pre-training, we adopted efficient tokenization and tensor alignment to maximize GPU throughput, and an optimization strategy combining AdamW, cosine annealing, and an aggressive learning rate schedule with warmup to accelerate early convergence. During fine-tuning, we employed a two-phase approach consisting of Supervised Fine-Tuning and Direct Preference Optimization. Experimental results demonstrate that L-Diff achieves 30% lower loss during the early training steps and reduces training time to 28% compared to the vanilla transformer baseline. These results indicate that our design choices effectively improve both optimization efficiency and training stability under resource-constrained conditions. For future work, we plan to extend L-Diff to models exceeding 10B parameters and evaluate its scalability on large-scale corpora such as FineWeb to further validate its effectiveness in real-world, high-performance language modeling scenarios.

References

- Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. 2025. [A survey on mixture of experts in large language models](#). *IEEE Transactions on Knowledge and Data Engineering*, 37(7):3896–3915.
- Tri Dao. 2023. [Flashattention-2: Faster attention with better parallelism and work partitioning](#). *Preprint*, arXiv:2307.08691.
- Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. 2025. [Liger kernel: Efficient triton kernels for llm training](#). *Preprint*, arXiv:2410.10989.

- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. [Lora: Low-rank adaptation of large language models](#). *Preprint*, arXiv:2106.09685.
- Yuxuan Kuang, Junjie Ye, Haoran Geng, Jiageng Mao, Congyue Deng, Leonidas Guibas, He Wang, and Yue Wang. 2024. [Ram: Retrieval-based affordance transfer for generalizable zero-shot robotic manipulation](#). *Preprint*, arXiv:2407.04689.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. [Fast inference from transformers via speculative decoding](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR.
- Wing Lian, Guan Wang, Bleys Goodson, Eugene Pentland, Austin Cook, Chanvichet Vong, and "Teknum". 2023. [Slimorca: An open dataset of gpt-4 augmented flan reasoning traces, with verification](#).
- Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. 2023. [Orca: Progressive learning from complex explanation traces of gpt-4](#). *Preprint*, arXiv:2306.02707.
- Aldo Pareja, Nikhil Shivakumar Nayak, Hao Wang, Krishnateja Killamsetty, Shivchander Sudalairaj, Wenlong Zhao, Seungwook Han, Abhishek Bhandwalidar, Guangxuan Xu, Kai Xu, Ligong Han, Luke Inglis, and Akash Srivastava. 2024. [Unveiling the secret recipe: A guide for supervised fine-tuning small llms](#). *Preprint*, arXiv:2412.13337.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct preference optimization: your language model is secretly a reward model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA. Curran Associates Inc.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. [Proximal policy optimization algorithms](#). *Preprint*, arXiv:1707.06347.
- Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, Madian Khabza, Han Fang, Yashar Mehdad, Sharan Narang, Kshitiz Malik, Angela Fan, Shruti Bhosale, Sergey Edunov, Mike Lewis, and 2 others. 2023. [Effective long-context scaling of foundation models](#). *Preprint*, arXiv:2309.16039.
- Lingling Xu, Haoran Xie, Si-Zhao Joe Qin, Xiaohui Tao, and Fu Lee Wang. 2023. [Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment](#). *Preprint*, arXiv:2312.12148.
- Tianzhu Ye, Li Dong, Yuqing Xia, Yutao Sun, Yi Zhu, Gao Huang, and Furu Wei. 2025. [Differential transformer](#). *Preprint*, arXiv:2410.05258.